

System Design Mastery

15 Real-World Scenarios Explained Simply

1

How does Hotstar handle millions of viewers during IPL matches without crashing?

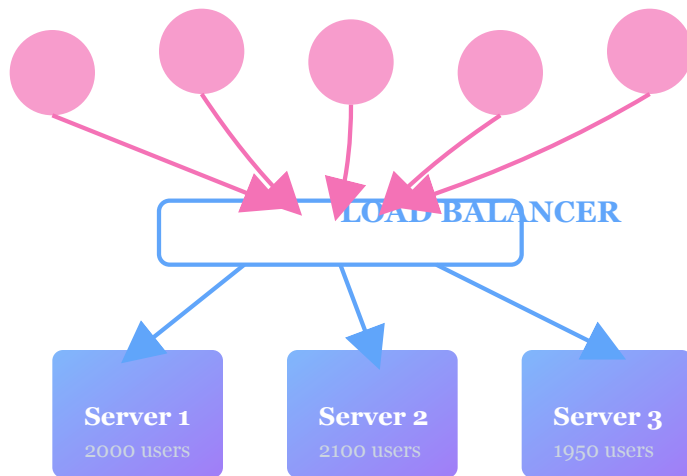
ANSWER: LOAD BALANCING & DISTRIBUTED SERVERS

Hotstar doesn't use just one server—it uses **thousands of servers spread across different locations**. When a match starts, traffic is automatically split among all servers so no single server gets overloaded.

- ▶ **Load Balancer:** Acts like a traffic cop, directing each user to the least busy server
- ▶ **Multiple Data Centers:** Servers in Mumbai, Delhi, Chennai etc., so users connect to the nearest one (faster + less load)
- ▶ **Auto-scaling:** If traffic spikes (IPL starts), system automatically spins up more servers
- ▶ **Caching:** Popular data (live score, video frame) is cached so servers don't recalculate



TRAFFIC DISTRIBUTION ACROSS SERVERS



2

Ticket sale scenario: 10,000 people trying to buy 1 seat simultaneously. How does the system ensure only 1 gets it?

ANSWER: DATABASE LOCKS & TRANSACTIONS

The system uses **database locks** to prevent race conditions. Think of it like a physical ticket booth with one cashier—only one person can complete their transaction at a time.

- ▶ **Atomic Transaction:** Check if seat is available → Reserve it → Update inventory (all or nothing)
- ▶ **Database Lock:** When first person starts buying, the database locks that seat temporarily
- ▶ **Others Wait:** Other 9,999 people's requests wait until lock is released
- ▶ **First Wins:** First person completes purchase → Seat marked sold → Lock releases
- ▶ **Others Fail:** When next person's request executes, seat is no longer available → transaction fails gracefully

ATOMIC TRANSACTION PROCESS

TIME SEQUENCE:

T0: 10,000 requests arrive simultaneously

T1: Person #1's transaction starts → Database LOCKS seat

✓ Check: Seat available? YES

✓ Reserve: Update DB → Seat owned by Person #1

✓ Confirm: Payment processed

→ LOCK RELEASED

T1-T1000: While Person #1 executes:

Person #2: Request arrives → WAIT (seat locked)

Person #3: Request arrives → WAIT (seat locked)

... (9,999 waiting)

T2: Next request executes (let's say Person #100's turn)

✗ Check: Seat available? NO (Person #1 owns it)

✗ Reject: "Sold out" message sent

→ Transaction rolled back, no changes

3

How does Uber show your driver's real-time location on the map (updates every 1-2 seconds)?

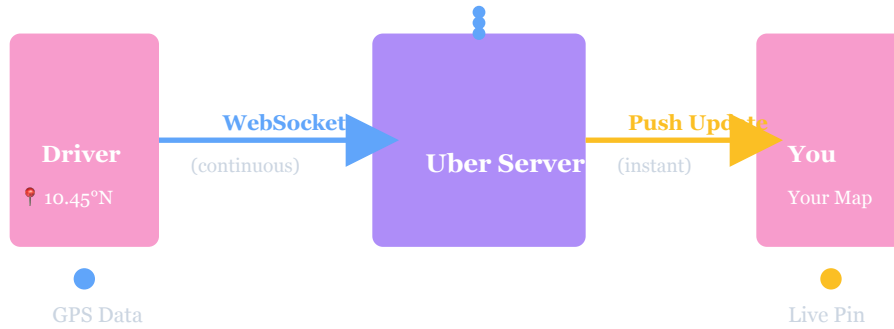
ANSWER: WEBSOCKETS & REAL-TIME MESSAGING

Instead of asking the server "where is driver?" every 1 second (wasteful), the driver's phone **continuously pushes location updates** to the server, which immediately broadcasts to your phone. It's like having an open phone line instead of calling back and forth.

- ▶ **WebSocket:** Open, persistent connection between driver's phone and server (not HTTP requests)
- ▶ **Location Push:** Driver's GPS sends new location every 1-2 seconds
- ▶ **Broadcast:** Server instantly sends this to your phone (and Uber's backend)
- ▶ **Low Latency:** Your map updates in real-time (no delay like HTTP polling would have)
- ▶ **Battery Efficient:** Connection stays open (fewer handshakes) vs repeated HTTP requests



REAL-TIME LOCATION BROADCASTING



Timeline:

T=0s: Driver at 10.450°N → Server receives → Your phone updates

T=1s: Driver at 10.451°N → Server receives → Your phone updates

T=2s: Driver at 10.452°N → Server receives → Your phone updates (repeat)

4

What is HTTP 429 "Too Many Requests" and why does a system send it?

ANSWER: RATE LIMITING TO PROTECT SERVICES

HTTP 429 is the server saying "slow down!" It happens when someone makes too many requests too fast. The server sends this to prevent abuse and protect itself from getting overwhelmed.

- ▶ **What triggers it:** Example: trying to login 100 times per second, or scraping a website rapidly
- ▶ **Rate Limiting Rule:** "Maximum 100 requests per minute per user" (or per IP address)
- ▶ **Counter Logic:** Server counts requests. If limit exceeded → sends 429 response
- ▶ **Retry-After Header:** Server includes "wait 60 seconds then try again" in response
- ▶ **Why it exists:** Prevents DDoS attacks, prevents accidental crashes from buggy clients, protects API quotas



RATE LIMITING IN ACTION

```
Request Scenario (limit: 10 req/minute):

Request #1 → ✓ OK (9 remaining)
Request #2 → ✓ OK (8 remaining)
Request #3 → ✓ OK (7 remaining)
...
Request #10 → ✓ OK (0 remaining)

Request #11 → ✗ HTTP 429 Too Many Requests
              "Retry-After: 60"
              (Come back in 60 seconds)

Request #12 → ✗ HTTP 429
Request #13 → ✗ HTTP 429
...
(wait 60 seconds)

Request #11 (after 60s) → ✓ OK (quota reset)
```

5

Netflix shows you personalized recommendations instantly. How does it handle millions of users' watch history?

ANSWER: CACHING & PRE-COMPUTATION

Netflix **doesn't calculate recommendations on-the-fly** for every user. Instead, it pre-computes popular recommendations and stores them in fast caches, so your phone gets results in milliseconds.

- ▶ **Batch Processing:** Every night, system analyzes all user watch data (millions of users) and generates recommendations
- ▶ **Cache Storage:** Results stored in Redis/Memcached (extremely fast in-memory database)
- ▶ **Instant Lookup:** When you open Netflix, recommendation is fetched from cache (not recalculated)

- ▶ **Periodical Updates:** Cache refreshes every 6-12 hours as new data comes in
- ▶ **CDN Distribution:** Cached data replicated across data centers so your nearest server has it

⚡ CACHING & RECOMMENDATION PIPELINE

BATCH PROCESSING (Night)

User A watches:
Action movies, Sci-fi → Algorithm: "Recommend Avengers, Dune"
(Millions of users processed simultaneously)

IN-MEMORY CACHE (Redis)

User A → [Movie1, Movie2, Movie3]
User B → [Movie4, Movie5, Movie6]
(Millions of entries, instant lookup)

Instant

YOUR NETFLIX APP

Fetch Recommendations

Response: ~50ms

🕒 Timeline:

Night (12 AM): Batch job runs, recommendations computed & cached

Day (3 PM): You open Netflix, recommendations fetched from cache instantly

6

How does a payment gateway ensure your bank transfer succeeds even if it crashes mid-transaction?

ANSWER: IDEMPOTENCY & TRANSACTION LOGS

The system **records every step of the transaction** before executing it. If it crashes, it can resume from where it left off, ensuring you're not charged twice and money doesn't disappear.

- ▶ **Idempotency Key:** Every transaction gets a unique ID. "Transfer ₹1000" with ID #12345
- ▶ **Pre-log Everything:** Before debiting your account, system writes "Step 1: Debit ₹1000" to permanent log
- ▶ **Execute:** Debit your account → Log shows "Step 1: Done"

- ▶ **Crash Recovery:** If server crashes, logs show exactly where transaction was: pending, in-progress, or completed
- ▶ **Idempotent Retry:** System won't execute "Debit ₹1000" again if ID #12345 is retried—it recognizes it already happened

CRASH-PROOF TRANSACTION FLOW

SCENARIO: User initiates transfer of ₹1000

STEP 1: Log Entry (Persistent Storage)

Transaction ID: #TXN_12345
Status: INITIATED
Amount: ₹1000
[Saved to disk immediately]

STEP 2: Pre-Check

✓ Verify user has balance
✓ Verify recipient exists
Status: VALIDATED
[Updated in log]

STEP 3: Debit Your Account

Your balance: ₹5000 → ₹4000
Status: DEBITED
[Log updated]

STEP 4: Credit Recipient

Recipient balance: ₹2000 → ₹3000
Status: CREDITED
[Log updated]

STEP 5: Complete

Status: SUCCESS
[Final log entry]

IF SERVER CRASHES between STEP 3 & 4:

Recovery on restart:
Read log → See ID #TXN_12345
Check status → "DEBITED" (in-progress)
Resume from STEP 4 → Credit recipient
✓ No double debit, transaction completes

IF SAME REQUEST RETRIED (duplicate):

Check ID #TXN_12345 in log
Status: "SUCCESS" (already completed)
→ REJECT retry, return success message
✓ Only charged once

How does Google Search show results in <0.5 seconds even though it searches billions of web pages?

ANSWER: INVERTED INDEXES & PRE-PROCESSING

Google doesn't search the entire internet in real-time. Instead, it **pre-processes all web pages offline** and creates an "inverted index"—a map of which pages contain each word. Searching this index is lightning-fast.

- ▶ **Inverted Index:** Map of "word → list of pages containing it" (like a book's index)
- ▶ **Offline Crawling:** Google crawlers continuously download web pages 24/7
- ▶ **Indexing:** Extract all words from each page, build the inverted index (happens once, stored permanently)
- ▶ **Query Time:** When you search "cat", system just looks up "cat" in index, gets list of pages instantly
- ▶ **Ranking:** From matching pages, rank by relevance using PageRank (pre-computed score)



INVERTED INDEX FOR FAST SEARCH

INVERTED INDEX (Built Offline):

```
Word "cat" →      Page1, Page5, Page99, Page150...
Word "dog" →      Page2, Page8, Page45, Page200...
Word "search" →   Page10, Page11, Page99...
```

QUERY TIME: User searches "cat"

- ✓ Lookup word "cat" in index → [Page1, Page5, Page99, Page150...]
- ✓ Retrieve top 10 from these pages (already ranked)
- ✓ Return results in ~0.3 seconds

WITHOUT INVERTED INDEX (Hypothetical):

```
Search "cat" → scan all 10 billion web pages
→ Find which ones have "cat"
→ Takes hours... ✗ Not feasible
```


8

Instagram shows your feed instantly even though it has 2 billion users. How?

ANSWER: FEED TIMELINE CACHING & PUSH MODEL

Instead of computing your feed on-the-fly each time you refresh, Instagram **pre-computes and caches your feed** in the background. When a friend posts, the system pushes the update to your cache.

- ▶ **Push Model:** When friend posts, system pushes it to all followers' cached feeds
- ▶ **Fan-Out:** If a celebrity posts, system distributes to millions of followers (across multiple servers)
- ▶ **Cache Hit:** When you refresh feed, it's already computed → served instantly
- ▶ **Trade-off:** Takes storage space (precomputing for 2B users), but read-time is fast
- ▶ **Selective Push:** For inactive users, feed computed on-demand (not precomputed)

FEED TIMELINE PUSH MODEL

YOUR CACHED FEED

[Post 1] Friend A: "Vacation pics"
[Post 2] Friend B: "Gym day"
[Post 3] Friend C: "Coffee"
(Updated in real-time)

WHEN FRIEND POSTS:

Friend D Posts: "Dinner"
→ Send to all followers' caches
→ Your cache: Add new post on top

Push

YOUR UPDATED FEED

[NEW Post 0] Friend D: "Dinner" ✨
[Post 1] Friend A: "Vacation pics"
[Post 2] Friend B: "Gym day"
[Post 3] Friend C: "Coffee"
(Instant refresh)

9

Amazon processes millions of orders per day without losing a single one. How is data durability guaranteed?

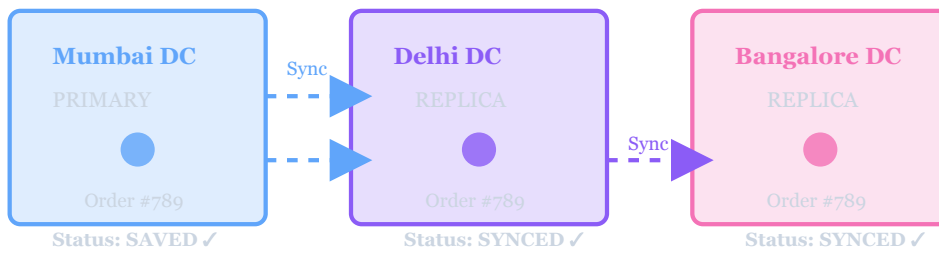
ANSWER: DATABASE REPLICATION & BACKUP STRATEGY

Amazon doesn't keep your order data in **one place**. Every order is replicated across multiple data centers. If one server fails, copies exist on others.

- ▶ **Primary Database:** Your order written to main database in Mumbai data center
- ▶ **Synchronous Replication:** Data immediately replicated to Delhi, Bangalore data centers
- ▶ **Acknowledgment:** Server only confirms order "saved" after all replicas are synced
- ▶ **Failover:** If Mumbai data center goes down, system switches to Delhi replica seamlessly
- ▶ **Backup to Storage:** Once daily, orders backed up to long-term storage (S3) for archival

ORDER REPLICATION ACROSS DATA CENTERS

ORDER DURABILITY



IF MUMBAI DC FAILS:

System detects Mumbai unreachable
→ Automatically switches to Delhi/Bangalore replicas
→ Customer's order still accessible, no data loss ✓

10

How does Twitter handle trending topics that change every minute without falling apart?

ANSWER: TIME-SERIES DATABASES & AGGREGATE COUNTERS

Twitter doesn't count tweets individually. Instead, it uses **counters that update in batches** and stores time-series data (tweets grouped by minute/hour). This allows quick trending calculations.

- ▶ **Time-Bucketing:** Tweet counts grouped by minute: "10:00-10:01 → 5000 tweets with #IPL"
- ▶ **Aggregate Counters:** Running total per minute (fast increment, no need to rescan all tweets)
- ▶ **Trending Algorithm:** Compare tweet velocity: "How fast did #IPL grow in last hour vs last week?"
- ▶ **Fast Lookup:** Trending query hits pre-computed metrics, not raw tweet data
- ▶ **Real-Time Updates:** Counters updated every 10-30 seconds, so trending list refreshes accordingly



TIME-SERIES TRENDING CALCULATION

TWITTER TRENDING LOGIC:

TIME SERIES DATA (Every minute):

10:00-10:01 → #IPL: 5000 tweets, #Cricket: 2000 tweets

10:01-10:02 → #IPL: 7000 tweets, #Cricket: 2500 tweets

10:02-10:03 → #IPL: 9000 tweets, #Cricket: 2800 tweets

CALCULATE VELOCITY (Growth Rate):

#IPL:

1-min growth: $7000/5000 = 1.4x$ (40% growth)

2-min growth: $9000/7000 = 1.28x$ (28% growth)

Average velocity: HIGH ↑

#Cricket:

1-min growth: $2500/2000 = 1.25x$ (25% growth)

2-min growth: $2800/2500 = 1.12x$ (12% growth)

Average velocity: MEDIUM

TRENDING RANKING:

1. #IPL (velocity 1.4x, fastest growing)

2. #Cricket (velocity 1.25x)

3. #Football (velocity 1.1x)

4. ...

WHY THIS IS FAST:

- ✓ Pre-computed counters (no rescan of all tweets)
- ✓ Math on metrics (division, not data processing)
- ✓ Results in microseconds

11

How does a video streaming platform buffer videos smoothly without constant rebuffering?

ANSWER: ADAPTIVE BITRATE STREAMING (ABR)

The video platform **continuously monitors your network speed** and automatically adjusts video quality on-the-fly. Fast network = high quality, slow network = lower quality (but smooth playback).

- **Multiple Qualities:** Same video encoded in 5 versions (480p, 720p, 1080p, 2K, 4K)

- ▶ **Bandwidth Detection:** Your phone measures how fast it's downloading chunks
- ▶ **Adaptive Selection:** If download speed = 5Mbps, select 720p version (matches speed)
- ▶ **Dynamic Switching:** Network slows down → automatically switch to 480p (lower bitrate, faster download)
- ▶ **Buffer Management:** Maintain 10-20 seconds of buffered video ahead (allows for brief network dips)



ADAPTIVE BITRATE STREAMING

VIDEO STREAMING QUALITY ADAPTATION

Available Video Files on Server:

480p 0.5 Mbps	720p 2.5 Mbps	1080p 5 Mbps	2K 8 Mbps	4K 15 Mbps
-------------------------	-------------------------	------------------------	---------------------	----------------------

Network Speed Monitoring:

T=0s: Network speed = 5.2 Mbps → Select 1080p ✓
 T=30s: Network speed drops to 2.0 Mbps → Switch to 720p ✓
 T=60s: Network recovers to 7 Mbps → Switch to 2K ✓

Result:

- ✓ Video never stops (quality adapts to network)
- ✓ Buffer maintained (10-20 seconds ahead)
- ✓ Smooth playback even on unstable connections

12

How does Google Maps show turn-by-turn directions instantly for any route in the world?

ANSWER: GRAPH PROCESSING & PRE-COMPUTED PATHS

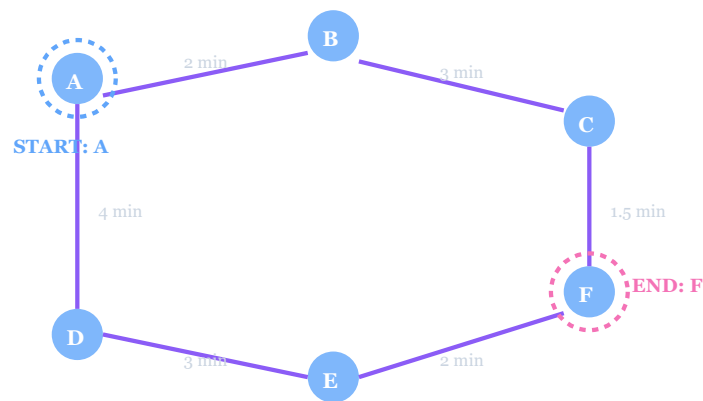
Google Maps treats roads as a **graph (nodes = intersections, edges = roads)**. Shortest path algorithms run on this graph, and popular routes are pre-computed and cached for instant lookup.

- ▶ **Map as Graph:** Every intersection = node, every road = edge with distance/time cost
- ▶ **Dijkstra/A* Algorithm:** Find shortest path between two nodes (optimized version runs in milliseconds)
- ▶ **Popular Routes Cached:** "Home → Office", "Mumbai → Delhi" pre-computed and stored
- ▶ **Real-Time Adjustments:** Traffic data updates (accidents, heavy traffic) → cost of edges changes → route recalculates
- ▶ **Hierarchical Graphs:** Fast highways processed separately from local roads (speeds up computation)



GRAPH-BASED ROUTE FINDING

MAP AS A GRAPH



SHORTEST PATHS ALGORITHM:

A → B (2 min) → C (3 min) → F (1.5 min) = 6.5 min

A → D (4 min) → E (3 min) → F (2 min) = 9 min ✗

✓ OPTIMAL

13

How does a bank's ATM system stay synchronized across thousands of machines so you can't withdraw money twice?

ANSWER: DISTRIBUTED TRANSACTIONS & CENTRAL BALANCE

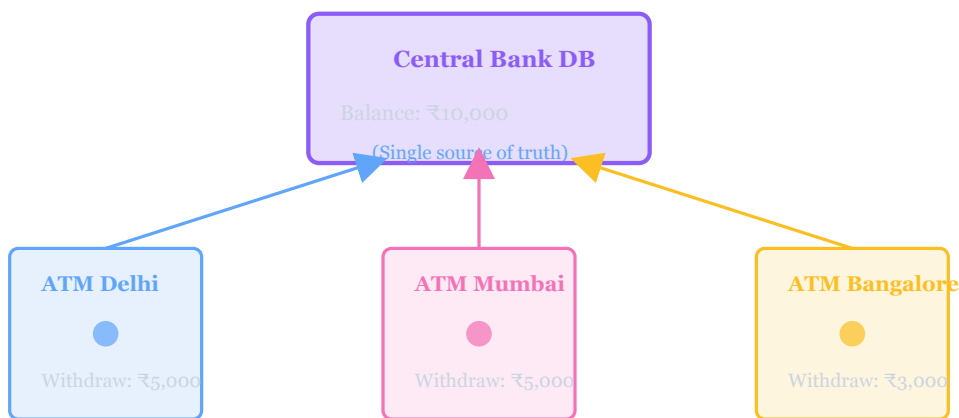
All ATMs connect to a **central bank database** that keeps your balance single-source-of-truth. Every withdrawal first checks this central balance, then deducts atomically.

- ▶ **Central Master Database:** Real-time balance stored in one secure location (replicated for backup)
- ▶ **Every ATM Queries First:** ATM in Delhi → "What's user balance?" → gets answer from Mumbai central DB
- ▶ **Atomic Deduction:** If balance is ₹10,000 and you withdraw ₹5,000, system atomically makes it ₹5,000
- ▶ **No Simultaneous Withdrawals:** If you try at 2 ATMs simultaneously, only first succeeds (second gets "insufficient balance")
- ▶ **Network Failure Handling:** If ATM loses connection, it doesn't dispense cash (safe failure —doesn't risk double withdrawal)



DISTRIBUTED ATM SYNCHRONIZATION

ATM NETWORK SYNCHRONIZATION



Scenario: ATM Delhi and Mumbai attempt withdrawal simultaneously

SEQUENCE:

```
T1: ATM Delhi requests balance
DB: ✓ Balance = ₹10,000 → LOCK (no other transaction allowed)

T1: ATM Mumbai requests balance simultaneously
DB: ✗ LOCKED, requests waits...
```

```
T2: ATM Delhi: Debit ₹5,000
    DB: Balance 10,000 → 5,000 (atomic update)
    DB: UNLOCK

T3: ATM Mumbai finally acquires lock
    DB: Check balance = ₹5,000 (updated)
    DB: ✗ Insufficient balance (only ₹5,000 left)
    Transaction rejected, card returns ✓

RESULT: Only ATM Delhi succeeds, no double withdrawal!
```

14

How does a chat application like WhatsApp ensure your message isn't lost even if the recipient is offline?

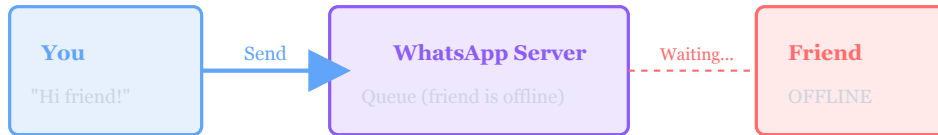
ANSWER: MESSAGE QUEUES & EVENTUAL DELIVERY

When you send a message, WhatsApp **stores it in a message queue** on the server. Once the recipient comes online, the server delivers the queued message automatically.

- ▶ **Send:** Your message → Server stores in queue with recipient's ID
- ▶ **Status: Sent (✓):** Message saved to queue (not delivered yet)
- ▶ **Recipient Offline:** Message waits in queue (for hours if needed)
- ▶ **Recipient Comes Online:** Server detects connection → pushes all queued messages instantly
- ▶ **Status: Delivered (✓✓):** Recipient receives message, confirms to server
- ▶ **Status: Read (✓✓🔵):** Recipient reads → notification sent back to sender

💬 MESSAGE QUEUE & OFFLINE DELIVERY

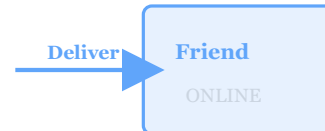
MESSAGE DELIVERY WITH OFFLINE SUPPORT



Status: Sent ✓

(Queued on server)

FRIEND COMES ONLINE:



Status: Delivered ✓✓

(Message transferred from queue to friend)

15

How does a system handle cascading failures when one service crashes and causes others to fail?

ANSWER: CIRCUIT BREAKER & GRACEFUL DEGRADATION

When one service crashes, systems use a **circuit breaker** to stop hammering it with requests. Instead, the system fails gracefully (shows cached data or simplified features) to prevent cascading failure across all services.

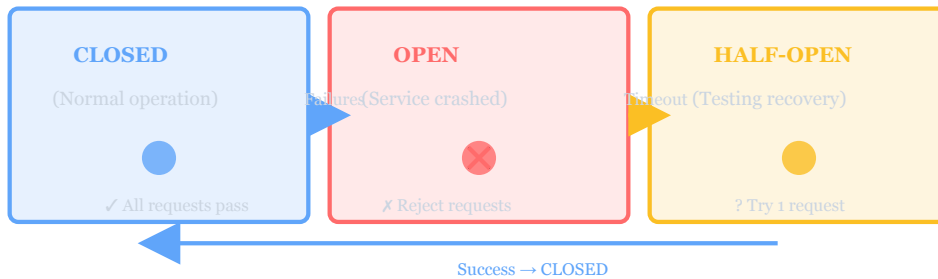
- ▶ **Circuit Breaker States:** Closed (normal) → Open (service down, stop requests) → Half-Open (test if recovered)
- ▶ **Fail-Fast:** When service A fails, don't wait for timeout—service B immediately fails and returns cached/fallback response
- ▶ **Graceful Degradation:** Instead of "Error", show "offline features"; e.g., YouTube loads video without recommendations
- ▶ **Timeout Protection:** Service B won't wait indefinitely for A's response; gives up after 5 seconds

- **Bulkheads:** Separate thread pools for each service, so one crash doesn't exhaust all threads



CIRCUIT BREAKER & CASCADING FAILURE PREVENTION

CIRCUIT BREAKER STATES



GRACEFUL DEGRADATION:

Scenario: Video streaming service fails (backend crash)

✗ Bad: App crashes entirely, shows "Server Error"

✓ Good: Circuit breaker opens → app shows cached video, hides recommendations

"Video ready. Recommendations unavailable (retry later)"

15 Real-World System Design Scenarios Explained Simply

Master these concepts to understand how the world's largest systems handle scale, concurrency, and resilience.